

# Architecture & Développement d'un Chat Multi-Salles

## 1. Contexte et Objectif Métier

L'objectif est de construire une application de messagerie instantanée permettant aux utilisateurs de rejoindre des salons de discussion thématiques.

Contrainte majeure de l'expérience utilisateur : si un utilisateur se déconnecte et revient, il doit pouvoir consulter l'historique récent de la discussion. Les messages doivent être persistés dans Redis pour permettre l'affichage d'un historique lors de la connexion.

## 2. Architecture du Projet

Pour valider ce prototype, vous devez implémenter une architecture trois tiers:

- **Le Client (React)** : Gère l'interface, la connexion aux salons et l'affichage des messages.
- **Le Serveur (Express + Socket.io)** : Joue le rôle de chef d'orchestre entre les clients et la base de données.
- **Le Stockage (Redis Stack)** : Centralise les données en exploitant la puissance en mémoire de Redis.

## 3. Mission à accomplir

### Étape 1 : Infrastructure (Docker)

Votre application nécessite des fonctionnalités avancées de Redis qui ne sont pas présentes dans l'image standard.

Vous devez configurer votre infrastructure pour supporter la manipulation de documents complexes et la recherche indexée.

- **Contrainte technique** : Modifiez votre script d'orchestration Docker pour utiliser l'image officielle regroupant les outils avancés de Redis (Redis Stack).
- **Ports requis** : Assurez-vous que votre conteneur expose le port standard de communication de données (données) et le port dédié à l'interface visuelle d'administration (Redis Insight).
- **Validation** : L'accès à l'interface graphique d'administration sur le port de visualisation doit être opérationnel avant de coder.

## Étape 2 : Modélisation NoSQL & Indexation (Backend)

Dans le répertoire dédié à vos schémas, vous devez définir la structure d'un message.

Un message est un objet contenant obligatoirement : l'identifiant du salon, l'identité de l'émetteur, le corps du texte et une empreinte temporelle.

- **Le piège de la recherche** : Pour pouvoir filtrer les messages par salon, Redis a besoin d'un index. Vous devez impérativement configurer le code de votre serveur pour qu'il **déclenche la création de cet index de recherche dès son démarrage**. Sans cela, toute tentative de récupération d'historique échouera.

## Étape 3 : Logique Temps Réel & Persistance (Socket.io & Redis)

Le serveur doit écouter et réagir à deux événements majeurs :

### Scénario A : Connexion à un salon (Aiguillage et Historique)

Lorsqu'un utilisateur demande à rejoindre une salle spécifique:

1. Le serveur doit isoler sa connexion dans le canal Socket.io correspondant à la pièce.
2. **Flux d'historique** : Le serveur doit immédiatement interroger Redis pour extraire **uniquement les 10 derniers messages** associés à cette pièce et les envoyer exclusivement à l'utilisateur qui vient de se connecter.

### Scénario B : Envoi d'un message (Transit et Persistance)

Lorsqu'un utilisateur transmet un message:

1. Le serveur réceptionne l'événement.
2. **Contrainte Redis** : Le serveur doit immédiatement sauvegarder ce message dans Redis en tant que document JSON.
3. Le serveur diffuse ensuite le message, mais **uniquement** aux membres connectés dans la pièce concernée.

## Étape 4 : Interface Utilisateur & Gestion des Flux (React)

Développez une interface client comprenant:

- Un écran initial pour saisir son pseudonyme et sélectionner un salon thématique (ex : "Général", "Tech", "Loisirs").
- Une zone d'affichage des messages du salon actuel.
- Un champ de saisie pour rédiger et envoyer un message.

**⚠ Sécurité et Cycle de vie (Critique)** : > Les connexions WebSockets sont persistantes. Vous devez utiliser les hooks de cycle de vie de React pour garantir que le client **ne crée qu'une seule connexion active**. Si l'utilisateur change de salon ou si le composant est re-

rendu, votre code doit impérativement nettoyer les anciens écouteurs d'événements et fermer proprement la connexion précédente.

#### 4. Défis d'Ingénierie (Pour aller plus loin)

1. **Indicateur de saisie (UX)** : Implémentez un mécanisme temps réel qui avertit le salon lorsqu'un utilisateur tape au clavier, en gérant l'extinction automatique du message après quelques secondes d'inactivité.
2. **Liste des membres en direct (Structure Redis)** : Utilisez une structure de données Redis de type **SET** (ensemble d'éléments uniques) pour stocker et mettre à jour la liste des utilisateurs connectés par salle. L'affichage doit se rafraîchir en temps réel côté client à chaque entrée/sortie.
3. **Audit de persistance (Preuve visuelle)** : Connectez-vous sur l'interface visuelle de votre instance Redis locale. Prenez une capture d'écran montrant comment vos messages sont structurés en mémoire sous forme de documents JSON et comment ils sont indexés. Joignez cette capture à votre rendu en expliquant la structure observée.